AI Analysis

Step 1:
04/08/2025 19:19
create a Singleton class called referee
I gave it access to no other files

After this prompt the AI went above what I asked it to do. It went ahead and started moving some of the code from Prog6 over without being prompted. While I did not ask it to do that it went ahead and did it. The AI did this step very well.

The Singleton code the AI created compared to the singleton code provided in class are very similar. As it should be expected because they are both SIngletons. They use very similar naming conventions as well. When comparing them the first difference that jumped out at me was in the constructor. In the sample code it gave a default value for myVariable, whereas in the AI did not create one. I believe this is simply due to the fact that it is not needed or is applicable in this case. Another major difference was in the code provided there were getters and setters. In the AI generated code there were no getters or setters for the Referee class.

04/08/2025 19:23
Move the PlayGame method to the refree class while having it still function as a Singleton
I gave it access to Prog6.java and Referee.java

Surprisingly the AI did this very well and I did not have to change anything else.

Step 2:
04/08/2025 19:25
Make the Dice.java class be a Singleton.
I gave it access to Dice.java

The AI did a good job at this task as well. Like before it followed the same format as the referee class and did not have getters or setters as well. With this Singleton class as well it did have more information in the constructor.

04/08/2025 19:26
Fix Player classes to work with the Singleton Dice class
I gave it access to all the Player classes but Player.java

When I gave the AI this prompt I thought it might struggle a little bit with it due the fact that it had to access several files at once and modify the roll and how the dice was initialized. I was happily surprised when it was able to complete this task with one prompt.

04/08/2025 19:27
Change the chance int to also work with the singleton

I gave it access to RandomPlayer.java

Unfortunately the AI struggled with this task. It kept on insisting it use the Math.Random function instead of using the perfectly good singleton Dice class I had created. Thus forcing this task to be accomplished manually.

04/08/2025 19:29
Make the int chance in the play method be a dice object from the singleton class with 2 sides.
I gave it access to UniquePlayer.java

Like before the AI also was unable to do this task. It kept running into the same problem. Therefore like last time I had to do this manually.

Step 3:
04/08/2025 19:33
Create an abstract RandomDice superclass. It should keep track of the nuber of slides of the Dice and have an abstract roll() method. The random object should stay with the Dice class.
I gave it access to Dice.java

This was a relatively easy task for the AI to complete and it did it very well. One thing that was extremely weird that happened when given this task. The AI decided to include an additional class declaration at the bottom of the code. It had the line public abstract class RandomDice { as the final line and as the first line. Not sure why it did that but was definitely an interesting thing.

Step 4:
04/08/2025 19:35
Create a FakeRandom subclass of RandomDice.
I gave it access to RandomDice.java

I wanted to break down this step into two portions because of what happened during the last step. I wanted to see if it would do the weird double class declaration. I wanted to see if it was an isolated instance, however that was not the case. It did it again for this class. Other than that it was able to create the class perfectly fine.

04/08/2025 19:36
Have the the roll be defined by a set of predetrimed numbers. Using a seed
I gave it access to FakeRandom.java

The AI took an interesting approach with this step. I thought that it might create a variable called seed and use that value the entire time. However it instead took seed as a parameter. I do like that the AI did this, while having it as a parameter is not needed. This does allow for multiple tests with different seeds to be executed at the same time.

Step 5:
04/08/2025 19:39
Create a new class called SevenPlayer that is the same as Fifteenplayer but rolls until 7
I gave it access to FifteenPlayer.java

There were zero issues with the AI completing this task. It was flawless; it even automatically created Javadoc.

Step 6:
04/09/2025 16:15
Create a FakeRandom test class using JUnit with testing to make sure that SevenPlayer works
I gave it access to SevenPlayer.

During this step I had to do a large amount of manual work. While the AI was able to create this file I had to go ahead and make several changes to the test ultimately resulting in me having to delete all the code that the AI generated. I had to do this for two reasons. The first was with the way the AI tried to implement JUnit. Due to the editor I was using I had to implement JUnit in a special way. The second was the test the AI generated were not JUnit tests they were a series of System.Out.Println lines and if statements that did not have any asserts.

To help with this step I created a FindSeed.Java class to help me find seed that would work for my tests.

Step 7:
There was no need for this step to be executed. My AI seems to have picked up the idea of always creating JavaDoc when creating code. It does this especially if it has access to another file that has used JavaDoc.

Reflection

One of the challenges I came across while doing this assignment was with JUnit. I have very little experience with JUnit. I have used it once for one test in a class long ago. Luckily you have provided us with sufficient resources to aid in my understanding so I could accomplish this assignment. Putting aside my lack of knowledge I encountered another issue, because I am using CoPilot for my AI I was using VisualStudio Code for my code editor. I had to take the time to implement the proper way to use JUnit in VisualStudio Code. This was the link I used if you have any curiosity on how I did that JUnit With VS Code. Once that was implemented I had to set up the testing environment slightly different from the example code you provided in class.

Time Management

The time I spent on this assignment was mainly in writing the report and setting up JUnit. When I first started implementing JUnit. I asked AI to generate me some tests and I looked at the tests you did in class as well. Both of the imports you had and the AI had were the same. However, these imports failed to work for me. So figuring out a solution to that took a large amount of my time. I spent roughly around 25 minutes on that process. Additionally I spent around 30 minutes creating this report, 20 minutes of debugging and testing, and finally it took about 25 minutes of prompting AI and generating code. This all comes to a grand total of and 1 hour and 45 minutes of work.

Conclusion

Overall this assignment was fairly straightforward to complete. The AI was able to complete a large majority of the assignment with little need for me to step in and complete tasks manually. I say this because if I had chosen a different editor there would have been two instances of me having to step in and fix the code. I do not fault the AI at all for the implementation of JUnit tasks due to the fact that it most likely sources that information from Eclipsed based JUnit tests.